Robot Movement Planning

Constraint Programming Project Report

Alessio Russo (mat. 376856)

1 Problem Statement

We consider a rectangular layer of boxes arranged on a discrete grid. Each box has dimensions $x \times y$, where the x-axis is aligned with the belt direction and the y-axis is orthogonal to it. The target packaging layer contains n boxes, whose destination positions are denoted by $(P_x[i], P_y[i])$, for $i = 1, \dots, n$. These positions are assumed to be non-overlapping and to tile a rectangular grid.

A robotic gripper with two parallel plates, each of size S, can pick a contiguous sequence of boxes along the x-axis. The total physical length of the sequence must be at most $S + x$, which corresponds to a maximum number of boxes equal to

$$\left\lfloor \frac{S + x}{x} \right\rfloor .$$

This formulation accounts for a maximum admissible overflow of $x/2$ on each side of the gripper.

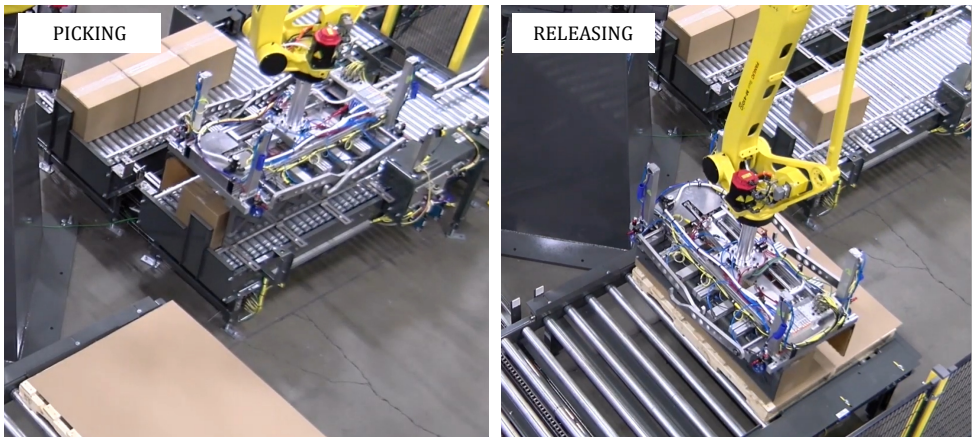


Figure 1: Example of a picking and releasing operations.

At each release operation, only one gripper plate can open. Therefore, the opening side must face a free direction, either the external boundary of the layer or a grid position that has not yet been filled. This constraint couples the geometric construction of groups with the temporal order in which they are released.

Figure 1 shows an example of the picking and releasing operations. The problem can therefore be decomposed into two related subproblems:

(A) **Grouping.** Partition the n boxes into groups that are contiguous along the x-axis, belong to the same row, and fit within the gripper capacity.

(B) **Scheduling.** Determine an ordering of the groups and assign an opening plate side to each group, so that the opening plate always faces a free direction at the time of release.

1.1 Parameters and Derived Constants

Based on this formulation, the MiniZinc model uses the following input parameters:

- the total number of boxes in the layer, denoted by `n_boxes`;
- the grid dimensions of the layer, denoted by `boxes_along_x` and `boxes_along_y`;
- the box dimension parallel to the gripper, denoted by `boxes_x_dim`. In the natural orientation, `boxes_x_dim` corresponds to x; if the layer is rotated, it corresponds to y;
- the gripper size S, denoted by `grip_size`.

These parameters are declared as follows:

```
par int: n_boxes;
par int: boxes_x_dim;
par int: boxes_along_x;
par int: boxes_along_y;
par int: grip_size;
```

From these inputs, the model derives three constants. First, the maximum number of boxes that can be picked in a single operation is computed directly from the gripper geometry:

$$\texttt{max\_pkble\_boxes} = \left\lfloor \frac{\texttt{grip\_size} + \texttt{boxes\_x\_dim}}{\texttt{boxes\_x\_dim}} \right\rfloor .$$

Second, the minimum number of groups required to cover one row is obtained by ceiling division:

$$\texttt{groups\_per\_row} = \left\lceil \frac{\texttt{boxes\_along\_x}}{\texttt{max\_pkble\_boxes}} \right\rceil .$$

Finally, the total number of groups is obtained by multiplying the number of groups per row by the number of rows:

$$\texttt{n\_groups} = \texttt{boxes\_along\_y} \cdot \texttt{groups\_per\_row}.$$

The corresponding declarations are:

```
par int: max_pkble_boxes = (grip_size + boxes_x_dim) div boxes_x_dim;

par int: groups_per_row = (boxes_along_x + max_pkble_boxes - 1)
                          div max_pkble_boxes;

par int: n_groups = boxes_along_y * groups_per_row;
```

2 Grouping

This part of the model admits several possible encodings. A natural but expensive formulation is a *fully enumerated* encoding, where each group explicitly stores the identifiers of all boxes it contains. Since groups may have different lengths, this representation usually requires a two-dimensional array with padding values for shorter groups.

```
array[1..n_groups, 1..max_pkble_packs] of var 0..n_packs: groups;
```

Here, the value 0 is used as a dummy value to represent an empty slot. This representation makes group membership explicit and easy to inspect. However, it introduces a large number of decision variables and requires additional constraints to enforce the structure of a valid solution. In particular, the model must ensure that each box appears exactly once, that non-empty positions inside each group are ordered, and that the groups are consistent with the geometric ordering of the layer.

A second possibility is a *binary assignment* encoding, where a Boolean variable states whether pack i belongs to group g. This representation can be written as follows:

```
array[1..n_groups, 1..n_packs] of var bool: belongs_to_group;
```

In this case, group membership is represented by the truth value of `belongs_to_group[g,i]`. Although this encoding is expressive, it is even larger than the fully enumerated representation. It also does not encode contiguity or same-row membership directly. These properties must be reconstructed through additional channeling constraints.

For this reason, the adopted model represents each group using its *start index* and *length*. An equivalent formulation could use the start and end indices instead. This interval-style representation is more compact because each group is described by only two main variables. Contiguity is implicit: once the start index and the length are known, the boxes in the group are uniquely determined. The same-row constraint can also be expressed directly by requiring the interval not to cross a row boundary. As a result, the model avoids explicit membership variables and reduces the number of constraints needed to describe valid groups.

```
array[1..n_groups] of var 1..n_packs: group_start;
array[1..n_groups] of var 1..max_pkble_packs: group_len;
```

The adopted encoding uses two arrays: `group_start`, which stores the first box of each group, and `group_len`, which stores the number of boxes in that group.

```
constraint group_start[1] = 1;

constraint forall(g in 1..n_groups - 1)(
  group_start[g + 1] = group_start[g] + group_len[g]
);

constraint group_start[n_groups] + group_len[n_groups] - 1 = n_packs;

constraint forall(g in 1..n_groups)(
  (group_start[g] - 1) div packs_along_x =
  (group_start[g] + group_len[g] - 2) div packs_along_x
);
```

The first three constraints define the groups as consecutive intervals over the row-major ordering of the packs. The first group starts from pack 1, and each following group starts immediately after the last pack of the previous group. The final constraint in this block forces the last group to end at pack n. Together, these conditions ensure that all packs are covered exactly once, without gaps or overlaps.

The fourth constraint enforces the geometric feasibility of each group. Since the gripper can only pick boxes aligned along the x-axis, a group cannot cross from one row to the next. The constraint therefore compares the row index of the first pack in the group with the row index of the last pack. If the two indices are equal, the whole interval lies on the same row.

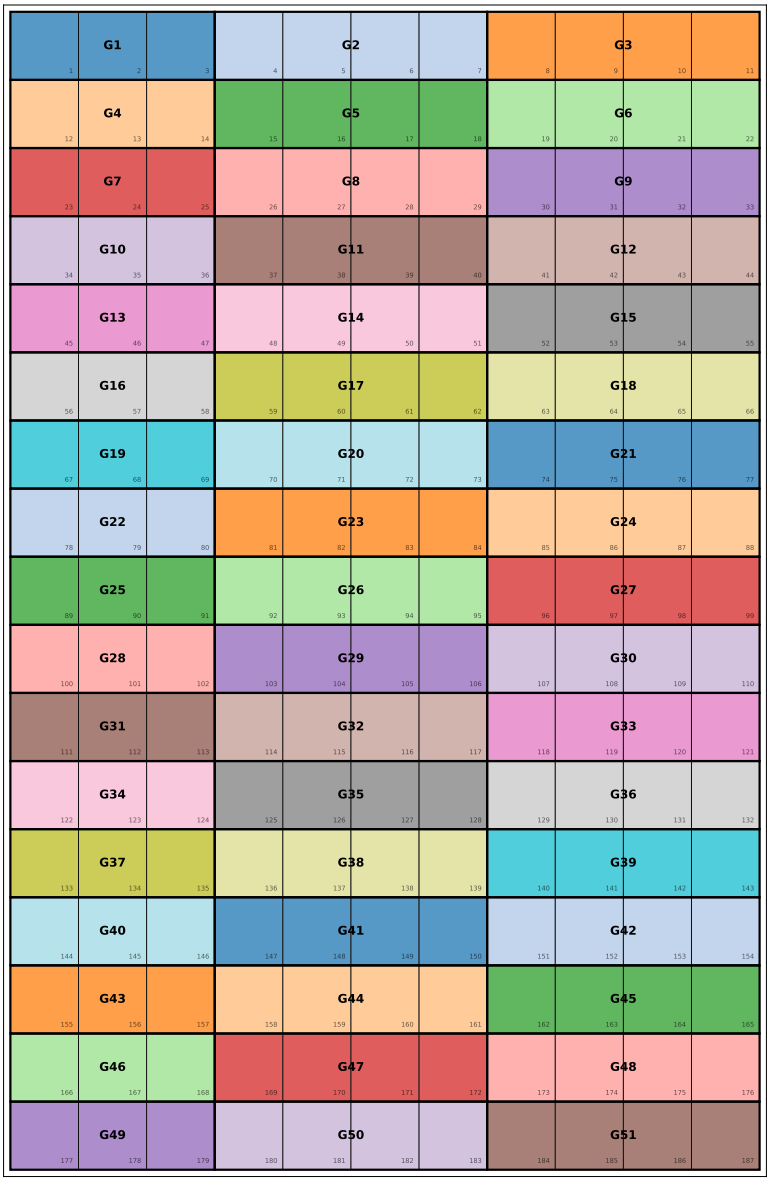


Figure 2: Pallet layout produced by the grouping stage: 51 groups tiling the 11×17 grid, each colored distinctly. Groups alternate 3–4–4 across each row.

3 Scheduling

The second part of the model determines the release order of the groups and selects which gripper plate opens for each release. In this case, the most direct representation consists in assigning to each group a single release position. The array `drop_order` stores the time step at which each group is released, while `plate` stores the plate opened for that release. The value 0 denotes the left plate, and the value 1 denotes the right plate.

```
array[1..n_groups] of var 1..n_groups: drop_order;
array[1..n_groups] of var 0..1: plate;
```

The only global constraint required to make `drop_order` a valid schedule is `alldifferent`. Since every value in `drop_order` belongs to the domain $1, \ldots,$ `n_groups`, forcing all values to be

different ensures that each release step is assigned to exactly one group.

```
constraint alldifferent(drop_order);
```

The feasibility of a release depends on the position of the adjacent groups in the same vertical column. As shown in Figure 2, groups are numbered from left to right within each row, and then from top to bottom across rows. Therefore, since each row contains `groups_per_row` groups, two groups that are aligned in consecutive rows have indices that differ by `groups_per_row`. For a generic group g, the vertical neighborhood is therefore given by $g \pm$ `groups_per_row`, whenever these indices are within the valid range. Groups in the first pallet row have no upper neighbor, while groups in the last pallet row have no lower neighbor.

The final constraint encodes the physical restriction on the gripper plates: when a group is released, the selected plate must open toward a free direction. A direction is free if the group lies on the corresponding pallet boundary, or if the group in that direction is scheduled to be released later and is therefore still absent from the pallet at the current release step.

Since the relevant neighbors are above and below the current group, the constraint is written by separating the first row, the internal rows, and the last row. This avoids invalid array accesses at the pallet boundaries.

```
constraint forall(g in 1..n_groups where g <= groups_per_row)(
  plate[g] = 0 \/
  (
    plate[g] = 1 /\
    drop_order[g + groups_per_row] > drop_order[g]
  )
);

constraint forall(g in 1..n_groups where
  g > groups_per_row /\ g <= n_groups - groups_per_row
)(
  (
    plate[g] = 0 /\
    drop_order[g - groups_per_row] > drop_order[g]
  )
  \/
  (
    plate[g] = 1 /\
    drop_order[g + groups_per_row] > drop_order[g]
  )
);

constraint forall(g in 1..n_groups where g > n_groups - groups_per_row)(
  (
    plate[g] = 0 /\
    drop_order[g - groups_per_row] > drop_order[g]
  )
  \/
  plate[g] = 1
);
```

The first constraint handles groups in the first pallet row. These groups have no upper neighbor, so opening plate 0 is always feasible. Opening plate 1, instead, is feasible only if the lower neighbor is released later.

The second constraint handles the internal rows. In this case, both vertical neighbors exist. Opening plate 0 is feasible only if the upper neighbor $g -$ `groups_per_row` is released later.

Similarly, opening plate 1 is feasible only if the lower neighbor $g + \texttt{groups\_per\_row}$ is released later.

The third constraint handles groups in the last pallet row. These groups have no lower neighbor, so opening plate 1 is always feasible. Opening plate 0 is feasible only if the upper neighbor is released later.

The model is a pure feasibility problem:

```
solve satisfy;
```

No objective function is required. Any assignment of `drop_order` and `plate` that satisfies the vertical-neighbor constraint defines a valid pick-and-place plan.

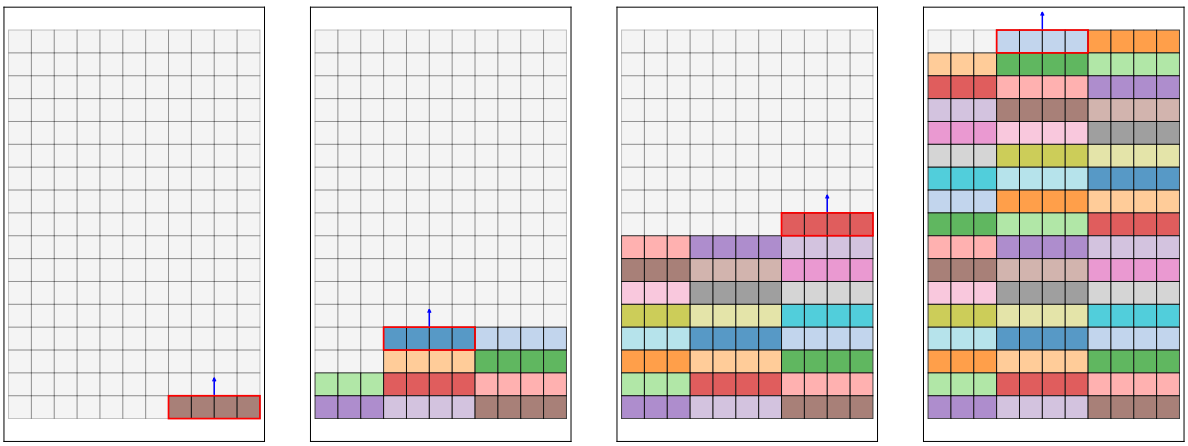


Figure 3: Example of a scheduling. The red border marks the group being placed; the blue arrow shows the opening plate side.

4 Calling the model from Python

Some small Python scripts are built around the MiniZinc model, which il loaded through `minizinc-python` and attached to the Gecode solver:

```
model    = Model("model.mzn")
solver   = Solver.lookup("gecode")
instance = Instance(solver, model)
```

The Python scripts assigns concrete values to the MiniZinc parameters and starts the solve:

```
instance["n_boxes"]        = pallet_info["n_boxes"]
instance["boxes_x_dim"]    = boxes_x_dim
instance["boxes_along_x"] = pallet_info["boxes_along_x"]
instance["boxes_along_y"] = pallet_info["boxes_along_y"]
instance["grip_size"]      = grip_size

result = instance.solve()
```

After solving, Python read the MiniZinc output arrays `group_start`, `group_len`, `drop_order`, and `plate` to get the pallet layout and schedule.

5 Benchmarking

The benchmark runs the MiniZinc model on every (pallet, pack, grip) combination from the lists in Table 1, against diffenrent solvers ("gecode", "cp-sat", "chuffed" and "highs", that uses different representation for the CSP problem, namely ..., with a per-run timeout of 300 seconds.

Pallet preset	Dimensions (mm)
Europallet	1200×800
Industrie	1200×1000
US 40×48	1219×1016
Half-pallet	800×600
Australian	1165×1165

Pack size (mm)
50×50
70×70
100×100
120×80
150×150
200×200

Grip multiplier
$1 \cdot$ `boxes_x_dim`
$2 \cdot$ `boxes_x_dim`
$3 \cdot$ `boxes_x_dim`

Table 1: Pallet presets, pack sizes, and gripper sizes used in the benchmark.

The gripper size is varied across the integer multiples $S \in \{1, 2, 3\} \cdot$ `boxes_x_dim`, in order to study the effect of the gripper size.

Figure 4 shows solve time with respect to the total number of boxes on the pallet for every solver across all instances. The arithmetic mean and min/max range per solver is summarised in Figure 5.

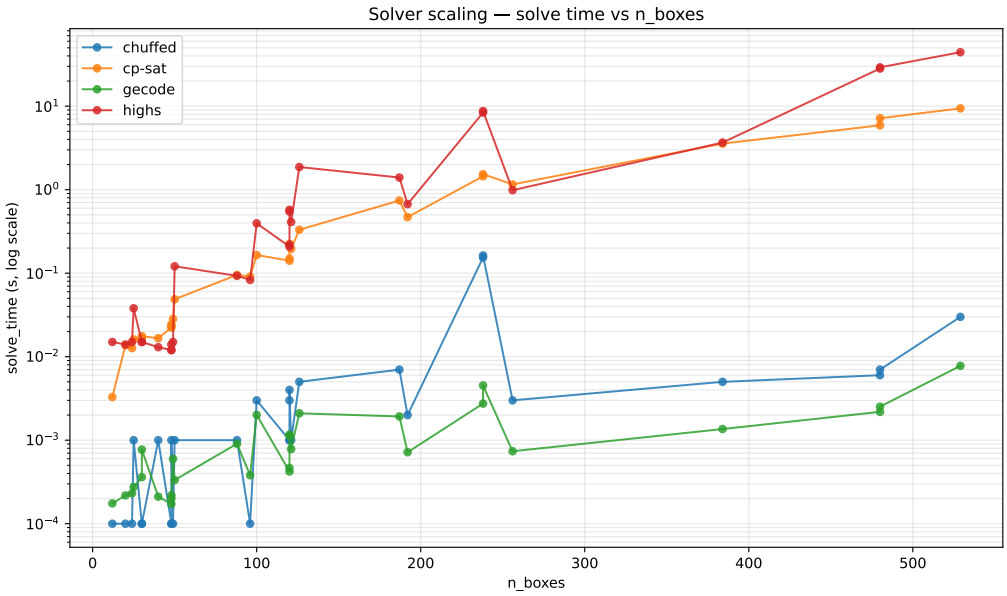


Figure 4: Solve time vs. total number of boxes — all solvers, all instances.

5.1 Per-pallet breakdown

Splitting the data by pallet preset (Figure 6) isolates the effect of pack count from pallet shape: for a fixed pallet, the curves are functions of pack count alone, while differences between pallets reflect the impact of `boxes_along_x` and `boxes_along_y` on the number of groups.

5.2 Effect of gripper size

Sweeping the gripper multiplier $k \in \{1, 2, 3\}$ changes `max_pkble_boxes` and therefore the total number of groups for a given pallet/pack pair. Smaller grippers force more groups per row, which

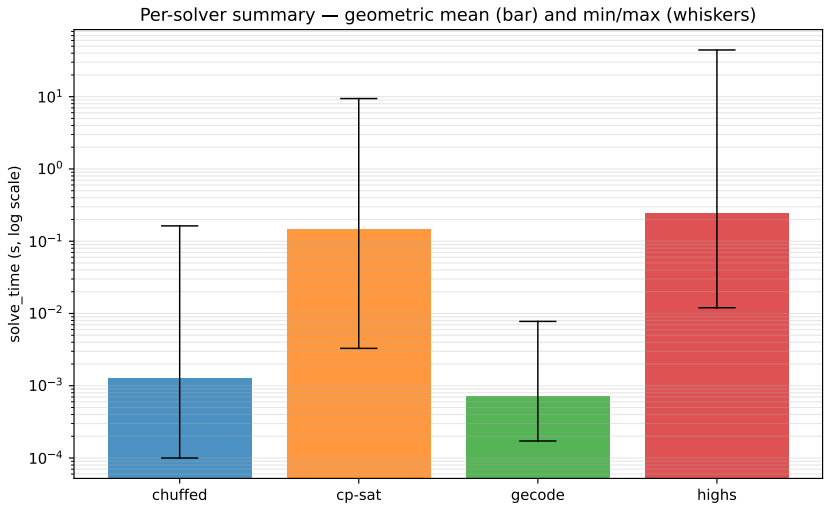


Figure 5: Per-solver mean of solve time (bar) with min/max whiskers across the full benchmark set.

tightens the partition but lengthens the schedule; larger grippers reduce the number of groups but loosen the row-partition constraint. Figure **??** reports the mean of solve time as a function of k, with one line per solver.

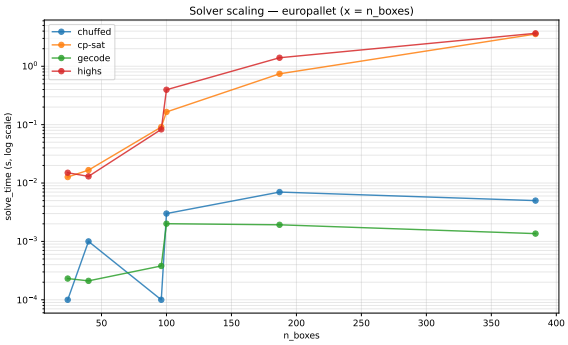
5.3 Wall time vs. solve time

The gap between Python-measured wall time and solver-reported solve time captures MiniZinc flattening and inter-process overhead. Figure 7 plots the two against each other; points above the diagonal indicate runs in which the harness cost dominated the search itself.

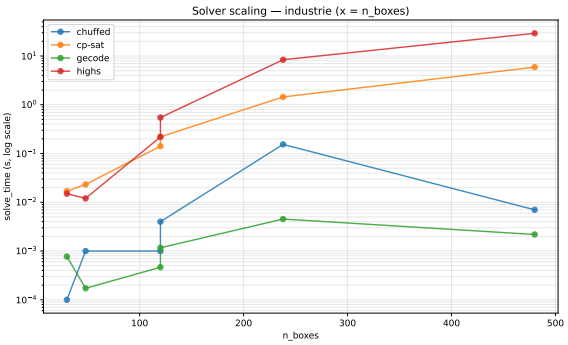
6 Conclusion

The two-stage formulation — grouping the pallet into pickable runs, then scheduling the drops with a plate side per group — maps cleanly onto MiniZinc decision variables and yields a compact, fully feasible model. The key modelling choice is to fix the number of groups statically and force the partition into a prefix-sum shape; this lets us pre-compute the neighbor structure as parameters rather than variables, keeping the plate-collision constraint a simple disjunction over indexed lookups.

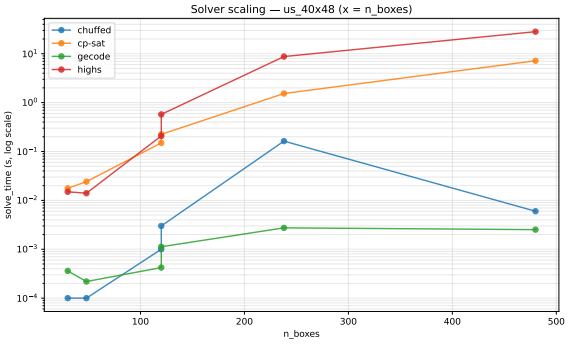
The benchmark confirms that the model scales well across realistic pallet sizes within the 5-minute budget, and exposes the expected per-solver trade-offs between flattening overhead and search performance.



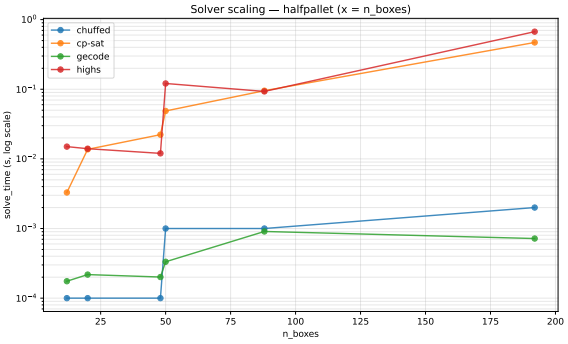
(a) Europallet



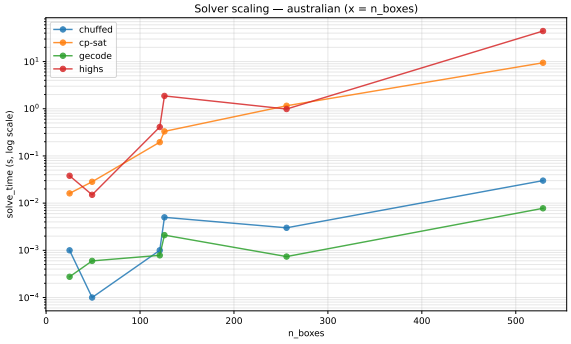
(b) Industrie



(c) US 40×48



(d) Half-pallet



(e) Australian

Figure 6: Solve time vs. number of boxes, broken down by pallet preset.

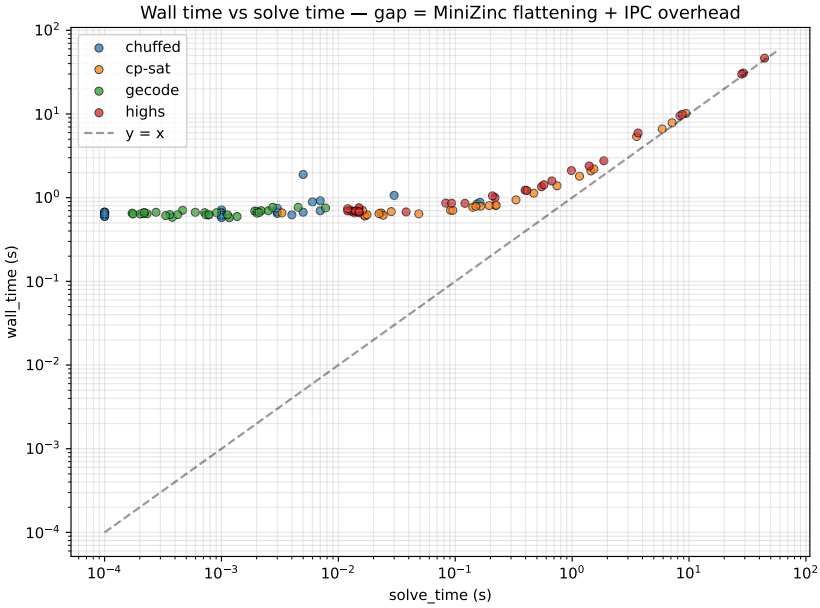


Figure 7: Wall time vs. solve time per run. The diagonal represents zero overhead; all points lie above it.